



S.G. Ganesh

How to Detect Integer Overflow

Integer overflows often result in nasty bugs. In this column, we'll look at some techniques to detect an overflow before it occurs.

Integer overflow happens because computers use fixed width to represent integers. So which are the operations that result in overflow? Bitwise and logical operations cannot overflow, while cast and arithmetic operations can. For example, ++ and += operators can overflow, whereas && or & operators (or even << and >> operators) cannot.

Regarding arithmetic operators, it is obvious that operations like addition, subtraction and multiplication can overflow.

How about operations like (unary) negation, division and mod (remainder)? For unary negation, $\sim MIN_INT$ is equal to MIN_INT (and not MAX_INT), so it overflows. Following the same logic, division overflows for the expression $(MIN_INT / -1)$. How about a mod operation? It does not overflow. The only possible overflow case $(MIN_INT \% -1)$ is equal to 0 (verify this yourself—the formula for % operator is $a \% b = a - ((a / b) * b)$).

Let us focus on addition. For the statement `int k = (i + j);`:

- (1) If *i* and *j* are of different signs, it cannot overflow.
- (2) If *i* and *j* are of same signs (- or +), it can overflow.
- (3) If *i* and *j* are positive integers, then their sign bit is zero. If *k* is negative, it means its sign bit is 1—it indicates the value of (*i* + *j*) is too large to represent in *k*, so it overflows.
- (4) If *i* and *j* are negative integers, then their sign bit is one. If *k* is positive, it means its sign bit is 0—it indicates that the value of (*i* + *j*) is too small to represent in *k*, so it overflows.

To check for overflow, we have to provide checks for conditions (3) and (4). Here is the straightforward conversion of these two statements into code. The function `isSafeToAdd` returns true or false after checking for overflow.

```
/* Is it safe to add i and j without overflow?
Return value 1 indicates there is no overflow;
else it is overflow and not safe to add i and j */
int isSafeToAdd(int i, int j) {
    if ((i < 0 && j < 0) && k >= 0) ||
        (i > 0 && j > 0) && k <= 0)
        return 0;
    return 1; // no overflow - safe to add i and j
}
```

Well, this does the work, but is inefficient. Can it be improved? Let us go back and see what *i* + *j* is, when it overflows.

If $((i + j) > INT_MAX)$ or if $((i + j) < INT_MIN)$, it overflows. But if we translate this condition directly into code, it will not work:

```
if (((i + j) > INT_MAX) || ((i + j) < INT_MIN))
    return 0; // wrong implementation
```

Why? Because (*i* + *j*) overflows, and when its result is stored, it can never be greater than INT_MAX or less than INT_MIN . That's precisely the condition (overflow) we want to detect, so it won't work.

How about modifying the checking expression? Instead of $((i + j) > INT_MAX)$, we can check the condition $(i > INT_MAX - j)$ by moving *j* to the RHS of the expression. So, the condition in `isSafeToAdd` can be rewritten as:

```
if ((i > INT_MAX - j) || (i < INT_MIN - j))
    return 0;
```

That works! But can we simplify it further? From condition (2), we know that for an overflow to occur, the signs of *i* and *j* should be different. If you notice the conditions in (3) and (4), the sign bit of the result (*k*) is different from (*i* and *j*). Does this strike you as the check that the ^ operator can be used? How about this check:

```
int k = (i + j);
if ((i < 0 && k < 0) || (i > 0 && k > 0))
    return 0;
```

Let us check it. Assume that *i* and *j* are positive values and when it overflows, the result *k* will be negative. Now the condition $(i \wedge k)$ will be a negative value—the sign bit of *i* is 0 and the sign bit of *k* is 1; so ^ of the sign bit will be 1 and hence the value of the expression $(i \wedge k)$ is negative. So is the case for $(j \wedge k)$ and when the & of two values is negative; hence, the condition check with < 0 becomes true when there is overflow. When *i* and *j* are negative and *k* is positive, the condition again is < 0 (following the same logic described above).

So, yes, this also works! Though the *if* condition is not very easy to understand, it is correct and is also an efficient solution! 

About the author:

S G Ganesh is a research engineer in Siemens (Corporate Technology). His latest book is "60 Tips on Object Oriented Programming", published by Tata McGraw-Hill. You can reach him at sgganesh@gmail.com.